Lab 2

CEG 3155 - Digital systems II

Fall 2025

School of Computer Engineering and Computer Science

University of Ottawa

Student Name and number: Mann Patel 300363879

Student Name and number: Justin Scaffidi 300383380

Submission Date Dec 2nd 2025

Group: 4

Table of content

Introduction ..... -3-
Explaining each file ..... -4-
Breakdown of internal functionning of important files ..... -6-
    UART Transmitter ..... -6-
    UART Receiver ..... -7-
    Message Holder ..... -7-
    Message Writer ..... -8-
Tool ..... -8-
Simulating ..... -9-
VHDL Simulations ..... -9-
    UART loopback test ..... -9-
    Lights with uart loopback ..... -10-
Conclusion and Problems ..... -11-
VHDL Code ..... -12-
    VHDL we wrote ..... -12-

Introduction

In this lab, we were tasked to create a VHDL UART implementation, featuring both a transmitter and receiver. This project is critical since it forces us to understand one of the most popular communication protocols used today as well as be able to create an implementation of it. It also gives us firsthand experience with digital design in the realm of digital communications.

We are building upon the lab 3 project, which was the traffic light system. We want to send a message on UART when we enter a new traffic light state. The message will be in the format of “Mg_Sr” saying that mainstreat is green and sidestreet is red. To solve this, we will be using two components, one will be listening to traffic light state changes and will be interfacing with the UART transmitter to feed the characters that need to be sent. We then have a UART transmitter which just takes data and sends it over the UART bus. There is also a UART receiver that is not connected to anything.

The UART transmitter contains a simple FSM where it waits for data to be written to its buffer, then will send it bit by bit, then wait for more data. The component we use to determine the data to send contains 4 FSMs, one for each UART packet. They are reset unless the traffic light is in the state that they represent. They will then go through their data and get stuck in the final state once it’s done emitting. Then when the state changes it gets reset back to the starting state.

Explaining each file

baud_generator.vhdl	This file takes in a 25mhz clock and divides it based on the SEL input to output a baud and baud8x, used for clocking the transmitter and receiver.
clk_div.vhdl	This file divides the 25mhz clock into a 1hz clock used by the traffic light system for handling state changes. It is built using the n_clk_divider since it worked better for simulation than the behavioural clk_div provided by the clock
counter.vhdl	This file counts up every clock and outputs when it reaches a specified target value
debuggableTrafficLightController.vhdl	This is the core of the project. It hooks together a UART system with the trafficLightController from lab 3 using a message_writer which will listen for traffic light state changes and uart transmissions to send the correct message for each state
message_holder.vhdl	Generic component that takes in a 6char ascii message and will output the current character, incrementing with each clock and then ultimately stalling in a final done state where it stays until it gets reset
message_writer.vhdl	Contains 4 message holders, one for each traffic light state, it will hold them reset until the traffic light is in the corresponding state. I use it with TDRE as a clock to send messages over UART
trafficLightController.vhdl	Lab3 solution for managing traffic lights, see previous lab report for more
hex_bcd_decoder.vhdl	Converts a hex value to decimal BCD values
load_default_state.vhdl	Used to load default states, not used in the new components because I was using VHDL 2008 NOR reductions, however quartus II from 2011 doesn't work with VHDL2008 even though there is an option for it so I had to rewrite it to just normal NOR expressions. It was quicker than using the component I made...
n_clk_divider.vhdl	Simple clock divider using a 1 hot counter. It is better to stack a ton of dividers than to make one big one... for memory reasons. Ideally it would use an actual ripple adder, but that is more complex and this works fine. Be wary of clock drift though.
nbit_comparator.vhdl	Compares values, not too spicy
nbit_register.vhdl	Self-explanatory, not too spicy
quad_mux_selector.vhdl	Single bit mux, using 1hot encoding for the select bits instead of BCD
ripple_adder.vhdl	Ripple adder, self explanatory
srlatch.vhdl	self explanatory

uart.vhdl	Combines the uart_receiver, uart_transmitter and baud_generator into one component that you can easily import and use. Just tidies things up
uart_receiver.vhdl	Handles the receiving of UART messages, checks for errors and puts them in RDR. Also provides the RDRF signal
uart_transmitter.vhdl	Handles sending UART messages, it takes data from TDR and provides a TDRE signal
enarjff.vhdl	Modifier version of the provided jkFF which adds reset and enable so I can actually use it in something and it's not completely useless
enardFF_2.vhdl	Given to us, d-flipflop implementation
onebitadder.vhdl	Given to us
onebitcomparator.vhdl	Given to us
dec_7seg.vhdl	Given to us, converts a BCD value to be rendered on a 7segment display

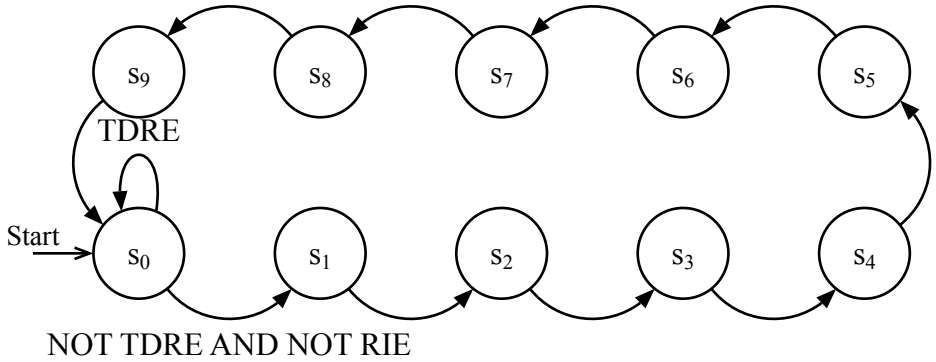
Breakdown of internal functionning of important files

UART Transmitter

The file starts in S0, waits for TDRE to be false, then will go through the FSM where state1 is the start bit, states 2-8 are date bits, state 9 is the parity bit, then goes back to the S0 as the stop bit

See this beautiful FSM

		NOT TDRE AND NOT RIE	TDRE
s0		s1	s0
s1	s2		
s2	s3		
s3	s4		
s4	s5		
s5	s6		
s6	s7		
s7	s8		
s8	s9		
s9	s0		



The TDRE signal is handled by a jk-flipflip which gets set when the FSM reaches s1 and reads from the TDR register, and it gets reset when someone writes to the TDR register. Oh also the output is flipped so all I said is in reverse (because reset needs to reset to empty)

Oh also there is TDRS which is when the Transmitter FSM State s1 reads from TDR, it puts it into TDRS, allowing other processes to write to TDR while the transmitter just does it's thing.

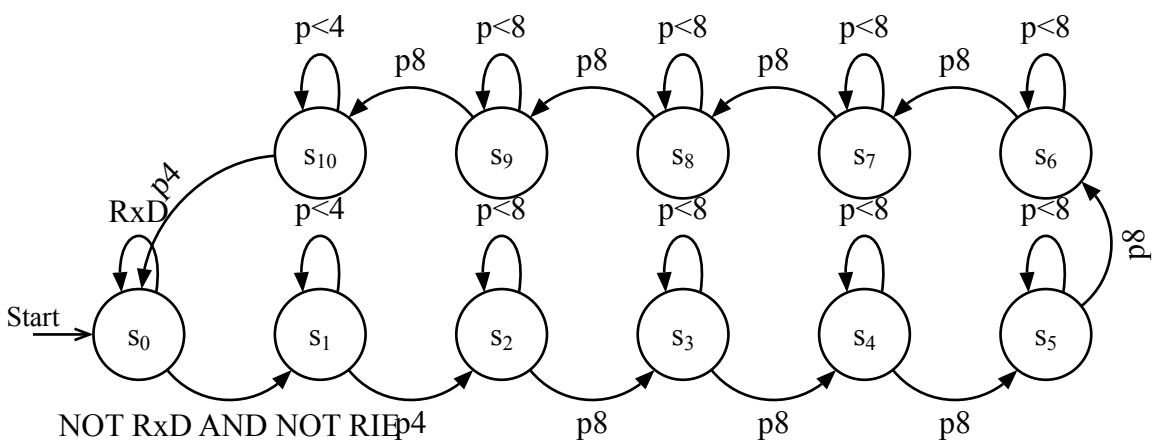
UART Receiver

This takes as input an 8x baud rate for sampling, and has a dual-FSM solution The first FSM is just one-hot counter for simplicity, it us used for counting the pulses. Since we want to read in the middle of the UART pulses, we need to offset and revert back to regular offsets at the end. We use this first FSM to count the clocks and depending on which state the other FSM is in, it will transition or wait.

For the 2nd FSM, it is very similar to the Transmission FSM, but it has one additional state for returning back to regular timing and for writing the error flags and such.

See this beautiful fsm (p = FSM1 count)

	NOT RxD AND NOT RIE	RxD	p4	p8	p<4	p<8
s0	s1	s0				
s1			s2		s1	
s2				s3		s2
s3				s4		s3
s4				s5		s4
s5				s6		s5
s6				s7		s6
s7				s8		s7
s8				s9		s8
s9				s10		s9
s10			s0		s10	



The RDRF signal is handled by a jk flipflop where state s10 loading sets the full status, and external sources reading reset the full status.

Oh also, there is the RDRS which is where the receiver FSM builds the input from the RxD line, and then it copies it into the RDR register in state s10 if it is valid.

Message Holder

This contains a simple FSM that just iterates until it reaches the end and stays there until it gets reset. Each FSM state will output a specific ASCII character in a 7bit buffer, or 0 when it reaches the end. It's very simple and powerful, especially when combined with the message_writer

Message Writer

Contains 4 instances of message_holder and keeps them in a reset state unless the traffic light is in its corresponding state, it will then send a pulse to the TDRE signal is triggered so it can write the next character.

Having it run using TDRE as a clock and holding everything in reset is very elegant since if the UART is too slow for the traffic light, it'll simply move onto the first character of the proper status. It will just truncate the previous message and continue with the up to date data. That way we don't have to worry about corruption or accruing an offset over time.

Tool

Tool is a progressive metal band that is known for their high mastery of their individual instruments. From Danny Carrey being a master drummer to James Maynard writing some of the most potent lyrics in metal, they are a force to be reconned with.

This however is a double edged sword as their fans have a reputation of thinking they're superior and worshipping the band as the gospel of god. This is not a joke, there is this thing called the holy gift which is when you liten to the tracks of their album Lateralus while following the fibonacci sequence for song selection. This has not been proven by the band as they also have a reputation of talking down on their fans and being rude to stupid questions. The theory of the holy gift is based on the fact that in the title track of the album, the verses follow the fibbonacci sequence when you look at the number of syllables they contain.

See:

```
(1) Black
(1) Then
(2) White are
(3) All I see
(5) In My infancy
(8) Red and yellow then came to be
-- Goes back down
(5) Reacihng out to me
(3) Lets me see
```

Jokes aside, idfk what to write in this section, is it for the max232 and oscilloscope part of the pre-project? Is it about our testing framework, I'll get into that later and idk why the pre-project should be on the final report ¯_(ツ)_/¯

Simulating

Since we need to simulate a fast clock because of the clock dividers, we need to disable a few of them to get some sort of meaningful output

In `clk_div`, disable the last clock divider that provides the `clock_1hz` signal, and map `clock_1hz <= clk_10khz;` so we have a slightly faster clock

We can now run the `debuggable_traffic_light_sim.vhdl`, either with ghdl and `make`, or by using modelsim. I don't know how modelsim works so YMMV, use common sense.

The simulation **WILL** take a long time to run (1-2min) depending on hardware, so be patient. The `wave.vcd` file is also going to be large (≈ 450mb).

I don't know if modelsim supports looking at nested structures, but if you open the `wave.vcd` file, navigate to `sim/debuggable_traffic_light_sim_inst` and add the signals.

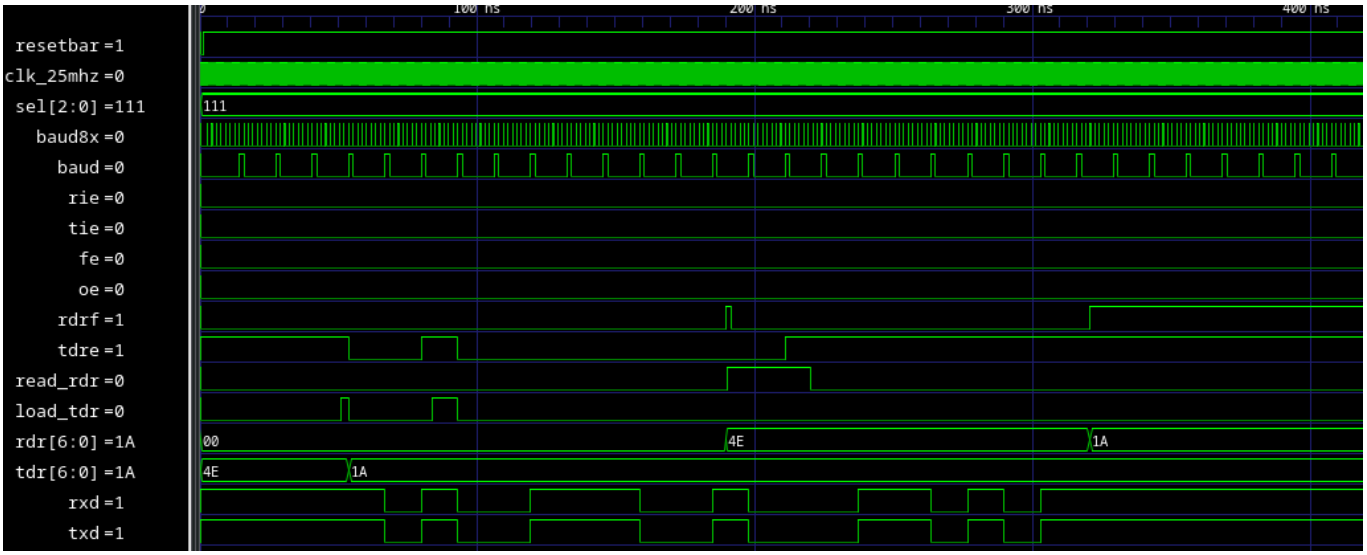
Since for the test suite I am looping TxD into RxD, you can see that the transmitter and receiver are properly working if you add the signal `sim/debuggable_traffic_light_sim_inst/debuggableTrafficLightController_inst/uart_inst/rdr`, you can see data being added as time goes

Other useful simulations which I did not include in this report are available in my testsuite.

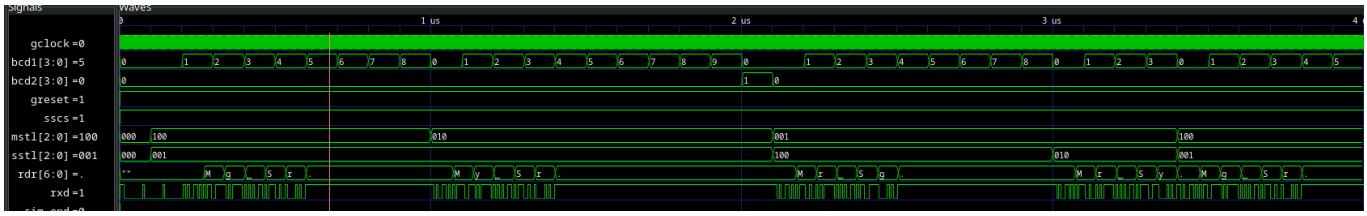
Individual components are not tested since they don't really make sense as standalones since you basically have to rewrite the other components to simulate them in any meaningful way. The end to end test shows that the behavior of the individual components is correct.

VHDL Simulations

UART loopback test



Lights with uart loopback



Conclusion and Problems

There were some issues encountered when going from raw-vhdl with ghdl to the lab environment.

To start off, I was under the false impression that Quartus supports VHDL2008, which is like so good omg it solves literally all my painpoints with VHDL… I digress.

Quartus II, an undocumented deprecated piece of software from 2013 claims the support, and has options for enabling it, but it just doesn't work. So I had to hastily do some compiler-assisted refactoring to make quartus understand it.

Next, it turns out I was using my simulation clock setup on the board, which made things run at very messed up speeds. This was fixed later.

Then it turns out I was sending my bits in reverse order, so I had to fix that, eventually I got most of it working on the board, but I was running into some decode problems with parity, so some characters were messed up… This eventually was all fixed ofc ofc and works perfectly now.

Oh also, the jk-flipflop provided by the prof does not have a reset and an enable. Please fix this, it makes it unuseable for a real solution.

However… I conclude, This was a very cool lab, it made us work with low-level protocols and by this time in the session, we have a good deal of useful components which we can use, making the development a lot more enjoyable. Lab 2 took a long time because there were so many complex things going on and we had to create our whole library from scratch, this was a lot more smooth sailing.

VHDL Code

VHDL we wrote

debuggableTrafficLightController.vhdl

```
library ieee;
use ieee.std_logic_1164.ALL;

-- VHDL 2008 required

entity debuggableTrafficLightController is
    port (
            -- in
            KeyboardClock: in std_logic; -- wtf is this for??
            KeyboardData: in std_logic; -- wtf is this for??
            SSCS: in std_logic;
            SW1: in std_logic_vector(3 downto 0);
            SW2: in std_logic_vector(3 downto 0);
            RxD: in std_logic;
            GClock: in std_logic; --  shove a 50mhz on this boi
            GReset: in std_logic;

            -- out
            TxD: out std_logic;
            MSTL: out std_logic_vector(2 downto 0);
            SSTL: out std_logic_vector(2 downto 0);
            BCD1: out std_logic_vector(3 downto 0);
            BCD2: out std_logic_vector(3 downto 0)
    );
end entity debuggableTrafficLightController;

architecture rtl of debuggableTrafficLightController is
    component trafficLightController is
        port (
                MSC : IN STD_LOGIC_VECTOR(3 downto 0);
                SSC : IN STD_LOGIC_VECTOR(3 downto 0);
                SSCS : IN STD_LOGIC;
                GClock : IN STD_LOGIC;
                GReset : IN STD_LOGIC;

                MSTL : OUT STD_LOGIC_VECTOR(2 downto 0);
                SSTL : OUT STD_LOGIC_VECTOR(2 downto 0);
                BCD1 : OUT STD_LOGIC_VECTOR(3 downto 0);
                BCD2 : OUT STD_LOGIC_VECTOR(3 downto 0)
            );
    end component;

    component message_writer is
    port (
```

```
            sel: IN STD_LOGIC_VECTOR(5 downto 0); -- MS(2 downto 0) SS(2 downto 0)
            clk: IN STD_LOGIC;
            resetBar: IN STD_LOGIC;
            buffer_free: IN STD_LOGIC;
            write_buffer: OUT STD_LOGIC;
            val: OUT STD_LOGIC_VECTOR(6 downto 0)
    );
    end component;

    component uart is
    port (
            resetBar: IN STD_LOGIC;
            clk_25mhz: IN STD_LOGIC;
            SEL: IN STD_LOGIC_VECTOR(2 downto 0);

            TDR: IN STD_LOGIC_VECTOR(6 downto 0);
            load_tdr: IN STD_LOGIC;
            TIE: IN STD_LOGIC;
            TDRE: OUT STD_LOGIC;

            RIE: IN STD_LOGIC;
            RDRF: OUT STD_LOGIC;
            RDR: OUT STD_LOGIC_VECTOR(6 downto 0);
            read_rdr: IN STD_LOGIC;
            OE: OUT STD_LOGIC;
            FE: OUT STD_LOGIC;

            TxD: OUT STD_LOGIC;
            RxD: IN STD_LOGIC
    );
    end component;


    component n_clk_divider is
        GENERIC (
                    DIVIDER_FACTOR: INTEGER := 2
                );
        port (
                 i_clk : in STD_LOGIC;
                 resetBar : in STD_LOGIC;
                 enable : in STD_LOGIC;
                 o_clk : out STD_LOGIC
             );
    end component;


    component clk_div IS
        PORT
        (
            clock_25Mhz       : IN  STD_LOGIC;
            GReset                  : IN  STD_LOGIC;
```

```
            clock_1Hz       : OUT STD_LOGIC);

    END component;

    signal tdr_free: std_logic;
    signal tdr_write: std_logic;
    signal tdr_val: std_logic_vector(6 downto 0);
    signal clk_25mhz: std_logic;
    signal clk_1hz: std_logic;

    signal clk_div_vec : std_logic_vector(3 downto 0);
    signal i_MSTL: std_logic_vector(2 downto 0);
    signal i_SSTL: std_logic_vector(2 downto 0);
begin
    controller: trafficLightController
     port map(
        MSC => SW1,
        SSC => SW2,
        SSCS => SSCS,
        GClock => clk_1hz,
        GReset => GReset,
        MSTL => i_MSTL,
        SSTL => i_SSTL,
        BCD1 => BCD1,
        BCD2 => BCD2
    );

    MSTL <= i_MSTL;
    SSTL <= i_SSTL;

    message_writer_inst: message_writer
     port map(
        sel(5 downto 3) => i_MSTL,
        sel(2 downto 0) => i_SSTL,
        clk => GClock,
        resetBar => GReset,
        buffer_free => tdr_free,
        write_buffer => tdr_write,
        val => tdr_val
    );

    uart_inst: uart
     port map(
        resetBar => GReset,
        clk_25mhz => clk_25mhz,
        SEL => "000",
        TDR => tdr_val,
        load_tdr => tdr_write,
        TIE => '0',
        TDRE => tdr_free,
        RIE => '0',
```

```
        RDRF => open,
        RDR => open,
        read_rdr => '0',
        OE => open,
        FE => open,
        TxD => TxD,
        RxD => RxD
    );

    -- clocks
    n_clk_divider_inst: n_clk_divider
     generic map(
        DIVIDER_FACTOR => 2
    )
     port map(
        i_clk => GClock,
        resetBar => GReset,
        enable => '1',
        o_clk => clk_25mhz
    );

    clk_div_inst: clk_div
     port map(
        clock_25Mhz => clk_25mhz,
        clock_1Hz => clk_1hz,
        GReset => GReset
    );


end architecture rtl;
```

uart.vhdl

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

entity uart is
    port (
            resetBar: IN STD_LOGIC;
            clk_25mhz: IN STD_LOGIC;
            SEL: IN STD_LOGIC_VECTOR(2 downto 0);

            TDR: IN STD_LOGIC_VECTOR(6 downto 0);
            load_tdr: IN STD_LOGIC;
            TIE: IN STD_LOGIC;
            TDRE: OUT STD_LOGIC;

            RIE: IN STD_LOGIC;
            RDRF: OUT STD_LOGIC;
            RDR: OUT STD_LOGIC_VECTOR(6 downto 0);
            read_rdr: IN STD_LOGIC;
            OE: OUT STD_LOGIC;
```

```
            FE: OUT STD_LOGIC;

            TxD: OUT STD_LOGIC;
            RxD: IN STD_LOGIC
    );
end entity uart;

architecture rtl of uart is
    component uart_transmitter is
        port (
            TDR: IN STD_LOGIC_VECTOR(6 downto 0);
            load_data: IN STD_LOGIC;
            resetBar: IN STD_LOGIC;
            baud: IN STD_LOGIC;
            TIE: IN STD_LOGIC;

            -- uart specific signals
            TxD: OUT STD_LOGIC;
            TDRE: OUT STD_LOGIC
        );
    end component;

    component uart_receiver is
        port (
                baud8x: IN STD_LOGIC;
                resetFull: IN STD_LOGIC;
                resetBar: IN STD_LOGIC;
                RIE: IN STD_LOGIC;
                RDRF: OUT STD_LOGIC;
                RDR: OUT STD_LOGIC_VECTOR(6 downto 0);
                OE: OUT STD_LOGIC;
                FE: OUT STD_LOGIC;
                RxD: IN STD_LOGIC
            );
    end component;

    component nbit_register IS
    generic (
                BITS : integer := 4
            );
    PORT (
             in_val : IN STD_LOGIC_VECTOR(BITS - 1 DOWNTO 0);
             in_load : IN STD_LOGIC;
             in_resetBar : IN STD_LOGIC;
             in_clock : IN STD_LOGIC;
             out_val : OUT STD_LOGIC_VECTOR(BITS - 1 DOWNTO 0)
         );
    END component;

    component baud_generator is
        port (
```

```
                sel: in std_logic_vector(2 downto 0);
                clk: in std_logic;
                resetBar: in std_logic;
                baud: out std_logic;
                baud_8x: out std_logic
            );
    end component;

    signal baud : STD_LOGIC;
    signal baud8x : STD_LOGIC;
begin

    baud_generator_inst: baud_generator
     port map(
        sel => SEL,
        clk => clk_25mhz,
        resetBar => resetBar,
        baud => baud,
        baud_8x => baud8x
    );

    uart_receiver_inst: uart_receiver
     port map(
        baud8x => baud8x,
        resetFull => read_rdr,
        resetBar => resetBar,
        RIE => RIE,
        RDRF => RDRF,
        RDR => RDR,
        OE => OE,
        FE => FE,
        RxD => RxD
    );

    uart_transmitter_inst: uart_transmitter
     port map(
        TDR => TDR,
        load_data => load_tdr,
        resetBar => resetBar,
        baud => baud,
        TIE => TIE,
        TxD => TxD,
        TDRE => TDRE
    );

end architecture rtl;
```

uart_transmitter.vhdl

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

```

```
-- VHDL 2008 required

entity uart_transmitter is
    port (
        TDR: IN STD_LOGIC_VECTOR(6 downto 0);
        load_data: IN STD_LOGIC;
        resetBar: IN STD_LOGIC;
        baud: IN STD_LOGIC;
        TIE: IN STD_LOGIC;

        -- uart specific signals
        TxD: OUT STD_LOGIC;
        TDRE: OUT STD_LOGIC
    );
end entity uart_transmitter;

architecture rtl of uart_transmitter is
    component nbit_register IS
    generic (
                BITS : integer := 4
            );
    PORT (
             in_val : IN STD_LOGIC_VECTOR(BITS - 1 DOWNTO 0);
             in_load : IN STD_LOGIC;
             in_resetBar : IN STD_LOGIC;
             in_clock : IN STD_LOGIC;
             out_val : OUT STD_LOGIC_VECTOR(BITS - 1 DOWNTO 0)
         );
    END component;

    component enarjff is
  PORT(
    i_j, i_k  : IN  STD_LOGIC;
    i_clock   : IN  STD_LOGIC;
        enable: IN STD_LOGIC;
        resetBar: IN STD_LOGIC;
    o_q, o_qBar : OUT STD_LOGIC);
    end component ;



    constant MAX_STATE: INTEGER := 9;
    signal nor_int_state: STD_LOGIC;
    signal int_state: STD_LOGIC_VECTOR(MAX_STATE downto 0);
    signal int_goto_state: STD_LOGIC_VECTOR(MAX_STATE downto 0);
    signal tdr_val : STD_LOGIC_VECTOR(6 downto 0);
    signal xor_tdrs_val : STD_LOGIC;
    signal tdrs_val : STD_LOGIC_VECTOR(6 downto 0);

    signal  i_tdre : std_logic;
    -- signals
    signal load_tdrs: std_logic := '0';
```

```
    signal reset_tdre: std_logic := '0';
begin

    states: nbit_register
     generic map(
        BITS => MAX_STATE+1
    )
     port map(
        in_val => int_goto_state,
        in_load => '1',
        in_resetBar => resetBar,
        in_clock => baud,
        out_val => int_state
    );

    tdr_reg: nbit_register
     generic map(
        BITS => 7
    )
     port map(
        in_val => TDR,
        in_load => load_data,
        in_resetBar => resetBar,
        in_clock => baud,
        out_val => tdr_val
    );

    tdrs_reg: nbit_register
     generic map(
        BITS => 7
    )
     port map(
        in_val => tdr_val,
        in_load => load_tdrs,
        in_resetBar => resetBar,
        in_clock => baud,
        out_val => TDRS_val
    );

    tdre_reg :  enarjff
    port map(
                i_j => load_data,
                i_k => reset_tdre,
                enable => '1',
                resetBar => resetBar,
                i_clock => baud,
                o_q => open,
                o_qBar => i_tdre
            );

    -- state transitions
```

```
    int_goto_state(0) <= nor_int_state OR int_state(9) OR (int_state(0) AND (TIE OR i_tdre));
    int_goto_state(1) <= int_state(0) AND NOT i_tdre AND NOT TIE;
    int_goto_state(2) <= int_state(1); -- write bit(1)
    int_goto_state(3) <= int_state(2); -- write bit(2)
    int_goto_state(4) <= int_state(3); -- write bit(3)
    int_goto_state(5) <= int_state(4); -- write bit(4)
    int_goto_state(6) <= int_state(5); -- write bit(5)
    int_goto_state(7) <= int_state(6); -- write bit(6)
    int_goto_state(8) <= int_state(7); -- write bit(7)
    int_goto_state(9) <= int_state(8); -- write xor

    -- signals
    load_tdrs  <= int_state(1);
    reset_tdre <= int_state(1);

    -- output driver
    nor_int_state <= NOT (
                      int_state(0) OR
                      int_state(1) OR
                      int_state(2) OR
                      int_state(3) OR
                      int_state(4) OR
                      int_state(5) OR
                      int_state(6) OR
                      int_state(7) OR
                      int_state(8) OR
                      int_state(9)
                      );
    xor_tdrs_val <= (
                     TDRS_val(6) XOR
                     TDRS_val(5) XOR
                     TDRS_val(4) XOR
                     TDRS_val(3) XOR
                     TDRS_val(2) XOR
                     TDRS_val(1) XOR
                     TDRS_val(0)
                     );

    TxD <= (nor_int_state) OR -- force 1 when in invalid state
         (int_state(0) AND '1') OR
         (int_state(1) AND '0') OR
         (int_state(2) AND TDRS_val(0)) OR
         (int_state(3) AND TDRS_val(1)) OR
         (int_state(4) AND TDRS_val(2)) OR
         (int_state(5) AND TDRS_val(3)) OR
         (int_state(6) AND TDRS_val(4)) OR
         (int_state(7) AND TDRS_val(5)) OR
         (int_state(8) AND TDRS_val(6)) OR
         (int_state(9) AND xor_tdrs_val);

    TDRE <= i_tdre;
```

```
end architecture rtl;
```

uart_receiver.vhdl

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- VHDL 2008 required

entity uart_receiver is
    port (
        baud8x: IN STD_LOGIC;
        resetFull: IN STD_LOGIC;
        resetBar: IN STD_LOGIC;
        RIE: IN STD_LOGIC;
        RDRF: OUT STD_LOGIC;
        RDR: OUT STD_LOGIC_VECTOR(6 downto 0);
        OE: OUT STD_LOGIC;
        FE: OUT STD_LOGIC;
        RxD: IN STD_LOGIC
    );
end entity uart_receiver;

architecture rtl of uart_receiver is
    component nbit_register IS
    generic (
                BITS : integer := 4
            );
    PORT (
             in_val : IN STD_LOGIC_VECTOR(BITS - 1 DOWNTO 0);
             in_load : IN STD_LOGIC;
             in_resetBar : IN STD_LOGIC;
             in_clock : IN STD_LOGIC;
             out_val : OUT STD_LOGIC_VECTOR(BITS - 1 DOWNTO 0)
         );
    END component;

    component enarjff is
  PORT(
    i_j, i_k  : IN  STD_LOGIC;
    i_clock   : IN  STD_LOGIC;
        enable: IN STD_LOGIC;
        resetBar: IN STD_LOGIC;
    o_q, o_qBar : OUT STD_LOGIC);
    end component ;

    signal pulse_counter : STD_LOGIC_VECTOR(7 downto 0);
    signal nor_pulse_counter: STD_LOGIC;
    signal pulse_counter_next : STD_LOGIC_VECTOR(7 downto 0);
    signal pulse_counter_reset: std_logic;
```

```
    constant MAX_STATE: INTEGER := 10;
    signal nor_int_state: STD_LOGIC;
    signal int_state: STD_LOGIC_VECTOR(MAX_STATE downto 0);
    signal int_goto_state: STD_LOGIC_VECTOR(MAX_STATE downto 0);

    -- signals
    signal read_rxd : STD_LOGIC;
    signal load_RDR : STD_LOGIC;
    signal load_rds : STD_LOGIC_VECTOR(7 downto 0);
    signal xor_rds_val : STD_LOGIC;
    signal rds_val : STD_LOGIC_VECTOR(7 downto 0);
    signal i_RDRF : STD_LOGIC;
    signal i_FE : STD_LOGIC;

    -- helpers
    signal rds_valid: STD_LOGIC;
begin

    pulse_counter_reg: nbit_register
     generic map(
        BITS => 8
    )
     port map(
        in_val => pulse_counter_next,
        in_load => '1',
        in_resetBar => resetBar,
        in_clock => baud8x,
        out_val => pulse_counter
    );

    nor_pulse_counter <= NOT
                         (pulse_counter(7) OR
                        pulse_counter(6) OR
                        pulse_counter(5) OR
                        pulse_counter(4) OR
                        pulse_counter(3) OR
                        pulse_counter(2) OR
                        pulse_counter(1) OR
                        pulse_counter(0));

    -- pulse counter since we have 8x baud. Useful for offsetting from the regular baud
    pulse_counter_next(0) <= pulse_counter(7) OR nor_pulse_counter OR pulse_counter_reset;
    pulse_counter_next(1) <= pulse_counter(0) AND NOT pulse_counter_reset;
    pulse_counter_next(2) <= pulse_counter(1) AND NOT pulse_counter_reset;
    pulse_counter_next(3) <= pulse_counter(2) AND NOT pulse_counter_reset;
    pulse_counter_next(4) <= pulse_counter(3) AND NOT pulse_counter_reset;
    pulse_counter_next(5) <= pulse_counter(4) AND NOT pulse_counter_reset;
    pulse_counter_next(6) <= pulse_counter(5) AND NOT pulse_counter_reset;
    pulse_counter_next(7) <= pulse_counter(6) AND NOT pulse_counter_reset;
```

```
    nbit_register_inst: nbit_register
     generic map(
        BITS => MAX_STATE + 1
    )
     port map(
        in_val => int_goto_state,
        in_load => '1',
        in_resetBar => resetBar,
        in_clock => baud8x,
        out_val => int_state
    );

    -- state machine
    nor_int_state <= NOT
                   (int_state(10) OR
                    int_state(9) OR
                    int_state(8) OR
                    int_state(7) OR
                    int_state(6) OR
                    int_state(5) OR
                    int_state(4) OR
                    int_state(3) OR
                    int_state(2) OR
                    int_state(1) OR
                    int_state(0));

    int_goto_state(0)  <= (int_state(10) AND pulse_counter(4)) OR (int_state(0) AND RxD) OR
    nor_int_state;
    int_goto_state(1)  <= (int_state(0)  AND NOT RxD AND NOT RIE) OR (int_state(1) AND NOT
    pulse_counter(4));
    int_goto_state(2)  <= (int_state(1)  AND pulse_counter(3)) OR (int_state(2)  AND NOT
    pulse_counter(7));
    int_goto_state(3)  <= (int_state(2)  AND pulse_counter(7)) OR (int_state(3)  AND NOT
    pulse_counter(7));
    int_goto_state(4)  <= (int_state(3)  AND pulse_counter(7)) OR (int_state(4)  AND NOT
    pulse_counter(7));
    int_goto_state(5)  <= (int_state(4)  AND pulse_counter(7)) OR (int_state(5)  AND NOT
    pulse_counter(7));
    int_goto_state(6)  <= (int_state(5)  AND pulse_counter(7)) OR (int_state(6)  AND NOT
    pulse_counter(7));
    int_goto_state(7)  <= (int_state(6)  AND pulse_counter(7)) OR (int_state(7)  AND NOT
    pulse_counter(7));
    int_goto_state(8)  <= (int_state(7)  AND pulse_counter(7)) OR (int_state(8)  AND NOT
    pulse_counter(7));
    int_goto_state(9)  <= (int_state(8)  AND pulse_counter(7)) OR (int_state(9)  AND NOT
    pulse_counter(7));
    int_goto_state(10) <= (int_state(9)  AND pulse_counter(7)) OR (int_state(10) AND NOT
    pulse_counter(4));

    -- signals
    read_rxd <= pulse_counter(7);
    load_RDR <= int_state(10) AND rds_valid AND pulse_counter(4);
    rds_valid <= rds_val(0) XNOR xor_rds_val;
    pulse_counter_reset <= int_state(0) OR (int_state(1) AND pulse_counter(4));
```

```
    xor_rds_val <=
                  rds_val(7) XOR
                  rds_val(6) XOR
                  rds_val(5) XOR
                  rds_val(4) XOR
                  rds_val(3) XOR
                  rds_val(2) XOR
                  rds_val(1);

    -- load data
    rds_reg: nbit_register
     generic map(
        BITS => 8
    )
     port map(
        in_val => load_rds,
        in_load => read_rxd,
        in_resetBar => resetBar,
        in_clock => baud8x,
        out_val => rds_val
    );

    load_rds(7) <= rds_val(6); -- left shift values in
    load_rds(6) <= rds_val(5);
    load_rds(5) <= rds_val(4);
    load_rds(4) <= rds_val(3);
    load_rds(3) <= rds_val(2);
    load_rds(2) <= rds_val(1);
    load_rds(1) <= rds_val(0);
    load_rds(0) <= RxD;


    -- transfer data to RRD
   rdr_reg: nbit_register
     generic map(
        BITS => 7
    )
     port map(
        in_val(6) => rds_val(1),
        in_val(5) => rds_val(2),
        in_val(4) => rds_val(3),
        in_val(3) => rds_val(4),
        in_val(2) => rds_val(5),
        in_val(1) => rds_val(6),
        in_val(0) => rds_val(7),
        in_load => load_RDR,
        in_resetBar => resetBar,
        in_clock => baud8x,
        out_val => RDR
    );
```

```
    rdrf_reg: enarjff
    port map(
                i_j => load_RDR,
                i_k => resetFull,
                enable => '1',
                resetBar => resetBar,
                i_clock => baud8x,
                o_q => i_RDRF,
                o_qBar => open
            );

    RDRF <= i_RDRF;
    i_FE <=NOT rds_valid OR NOT RxD;
    -- errors
    error_reg: nbit_register
     generic map(
        BITS => 2
    )
     port map(
        in_val(0) => i_RDRF, -- overrun error
        in_val(1) => i_FE, -- framing errror (parity fail or no clear stop bit)
        out_val(0) => OE,
        out_val(1) => FE,
        in_load => load_RDR,
        in_resetBar => resetBar,
        in_clock => baud8x
    );

end architecture rtl;
```

message_holder.vhdl

```
LIBRARY ieee;
use ieee.std_logic_1164.ALL;

-- holds strings of 6 ascii chars (generic this is hard)
entity message_holder is
    generic ( message: std_logic_vector(6*7 -1 downto 0));
    port (
        resetBar: IN STD_LOGIC;
        nextChar: IN STD_LOGIC;
        char: OUT STD_LOGIC_VECTOR(6 downto 0);
        done: OUT STD_LOGIC
    );
end entity message_holder;

architecture rtl of message_holder is
    component nbit_register IS
    generic (
                BITS : integer := 4
            );
```

```
    PORT (
            in_val : IN STD_LOGIC_VECTOR(BITS - 1 DOWNTO 0);
            in_load : IN STD_LOGIC;
            in_resetBar : IN STD_LOGIC;
            in_clock : IN STD_LOGIC;
            out_val : OUT STD_LOGIC_VECTOR(BITS - 1 DOWNTO 0)
        );
END component;

    signal int_state: std_logic_vector(6 downto 0);
    signal int_goto_state: std_logic_vector(6 downto 0);
    signal nor_int_state: std_logic;
begin

    states: nbit_register
     generic map(
        BITS => 7
    )
     port map(
        in_val => int_goto_state,
        in_load => '1',
        in_resetBar => resetBar,
        in_clock => nextChar,
        out_val => int_state
    );

    nor_int_state <= NOT
                    (int_state(6) OR
                    int_state(5) OR
                    int_state(4) OR
                    int_state(3) OR
                    int_state(2) OR
                    int_state(1) OR
                    int_state(0));
    -- chutes n ladders type FSM where you go up and fall all the way down when we reset
    int_goto_state(0) <= nor_int_state;
    int_goto_state(1) <= int_state(0);
    int_goto_state(2) <= int_state(1);
    int_goto_state(3) <= int_state(2);
    int_goto_state(4) <= int_state(3);
    int_goto_state(5) <= int_state(4);
    int_goto_state(6) <= int_state(5) OR int_state(6);

    g: for i in 6 downto 0 generate
        char(i) <= (int_state(5) AND message(i)) OR
                (int_state(4) AND message(i+7)) OR
                (int_state(3) AND message(i+7*2)) OR
                (int_state(2) AND message(i+7*3)) OR
                (int_state(1) AND message(i+7*4)) OR
                (int_state(0) AND message(i+7*5));
    end generate;
```

```
    done <= int_state(6);

end architecture rtl;
```

message_writer.vhdl

```
LIBRARY ieee;
use ieee.std_logic_1164.ALL;

-- VHDL 2008 required

entity message_writer is
    port (
              sel: IN STD_LOGIC_VECTOR(5 downto 0); -- MS(2 downto 0) SS(2 downto 0)
              clk: IN STD_LOGIC;
              resetBar: IN STD_LOGIC;
              buffer_free: IN STD_LOGIC;
              write_buffer: OUT STD_LOGIC;
              val: OUT STD_LOGIC_VECTOR(6 downto 0)
    );
end entity message_writer;

architecture rtl of message_writer is
    component nbit_register IS
    generic (
                BITS : integer := 4
            );
    PORT (
              in_val : IN STD_LOGIC_VECTOR(BITS - 1 DOWNTO 0);
              in_load : IN STD_LOGIC;
              in_resetBar : IN STD_LOGIC;
              in_clock : IN STD_LOGIC;
              out_val : OUT STD_LOGIC_VECTOR(BITS - 1 DOWNTO 0)
         );
    END component;

    component message_holder is
    generic ( message: std_logic_vector(6*7 -1 downto 0));
    port (
        resetBar: IN STD_LOGIC;
        nextChar: IN STD_LOGIC;
        char: OUT STD_LOGIC_VECTOR(6 downto 0);
        done: OUT STD_LOGIC
    );
    end component;



    component quad_mux_selector IS
    PORT (
    in_tt, in_tf, in_ft, in_ff : IN STD_LOGIC;
    in_sel_tt : IN STD_LOGIC;
```

```
    in_sel_tf : IN STD_LOGIC;
    in_sel_ft : IN STD_LOGIC;
    in_sel_ff : IN STD_LOGIC;
    out_mux : OUT STD_LOGIC
                        );
    END component;



    constant MSG_MSG_VAL: STD_LOGIC_VECTOR(6*7 -1 downto 0) :=
    "1001101110011110111111010011111001000010 10";
    constant MSY_MSG_VAL: STD_LOGIC_VECTOR(6*7 -1 downto 0) :=
    "1001101111100110111111010011111001000010 10";
    constant SSG_MSG_VAL: STD_LOGIC_VECTOR(6*7 -1 downto 0) :=
    "1001101111001010111111010011111001110001 010";
    constant SSY_MSG_VAL: STD_LOGIC_VECTOR(6*7 -1 downto 0) :=
    "1001101111001010111111010011111100100010 10";

    signal active_state: std_logic_vector(3 downto 0);
    signal bank_reset: std_logic_vector(3 downto 0);
    signal nor_state_done : std_logic;
    signal state_done: std_logic_vector(3 downto 0);
    signal read_next: STD_LOGIC;

    signal msg_out : std_logic_vector(6 downto 0);
    signal msy_out : std_logic_vector(6 downto 0);
    signal ssg_out : std_logic_vector(6 downto 0);
    signal ssy_out : std_logic_vector(6 downto 0);

    signal nor_int_state: std_logic;
    signal int_state: std_logic_vector(4 downto 0);
    signal int_goto_state: std_logic_vector(4 downto 0);
begin

    bank_reset(0) <= active_state(0) AND resetBar;
msg_bank_holder: message_holder
                  generic map( message => MSG_MSG_VAL)
                  port map(
                     resetBar => bank_reset(0),
                     nextChar => read_next,
                     char => msg_out,
                     done => state_done(0)
                 );

                 bank_reset(1) <=active_state(1) AND resetBar;
msy_bank_holder: message_holder
                  generic map( message => MSY_MSG_VAL)
                  port map(
                     resetBar => bank_reset(1),
                     nextChar => read_next,
                     char => msy_out,
                     done => state_done(1)
                 );
```

```
                  bank_reset(2) <=active_state(2) AND resetBar;
ssg_bank_holder: message_holder
                   generic map( message => SSG_MSG_VAL)
                   port map(
                      resetBar => bank_reset(2),
                      nextChar => read_next,
                      char => ssg_out,
                      done => state_done(2)
                  );

                  bank_reset(3) <=active_state(3) AND resetBar;
ssy_bank_holder: message_holder
                   generic map( message => SSY_MSG_VAL)
                   port map(
                      resetBar => bank_reset(3),
                      nextChar => read_next,
                      char => ssy_out,
                      done => state_done(3)
                  );

                  active_state(0) <= sel(5); -- MSG
                  active_state(1) <= sel(4); -- MSY
                  active_state(2) <= sel(2); -- SSG
                  active_state(3) <= sel(1); -- SSY

                  -- output
                  ii: for i in 6 downto 0 generate
                      quad_mux_selector_inst: quad_mux_selector
                       port map(
                          in_tt => ssy_out(i),
                          in_tf => ssg_out(i),
                          in_ft => msy_out(i),
                          in_ff => msg_out(i),
                          in_sel_tt => active_state(3),
                          in_sel_tf => active_state(2),
                          in_sel_ft => active_state(1),
                          in_sel_ff => active_state(0),
                          out_mux => val(i)
                      );
                  end generate;


                  -- states
                  states: nbit_register
                   generic map(
                      BITS => 5
                  )
                   port map(
                      in_val => int_goto_state,
                      in_load => '1',
```

```
                    in_resetBar => resetBar,
                    in_clock => clk,
                    out_val => int_state
                );

                nor_int_state <= NOT
                                (int_state(4) OR
                                int_state(3) OR
                                int_state(2) OR
                                int_state(1) OR
                                int_state(0));

                nor_state_done <= NOT
                                 (state_done(3) OR
                                 state_done(2) OR
                                 state_done(1) OR
                                 state_done(0));

                int_goto_state(0) <= nor_int_state OR int_state(3) OR (int_state(4) AND nor_state_done)
                OR (int_State(0) AND NOT buffer_free); -- it's 5am, this makes sense in my mind, good
                lucky figuring this out :)
                int_goto_state(1) <= (int_state(0) AND buffer_free);
                int_goto_state(2) <= int_state(1) OR (int_state(2) AND buffer_free);
                int_goto_state(3) <= int_state(2) AND nor_state_done;
                int_goto_state(4) <= int_state(4) AND NOT nor_state_done;

                -- signals
                write_buffer <= int_state(1);
                read_next <= int_state(3);

end architecture rtl;
```